

Informe Técnico de Desarrollo de Software: Proyecto TeKnical May Cry

Mattia Federico, Ferrari Ulises, Del Pino Lucas y Orue Máximo

Escuela de Educación Secundaria Técnica N.º 1 "Eduardo Ader", Vicente López

Laboratorio de Programación – 5.º 3.º A-B

Prof. Mansilla Muñoz York E. y Prof. Ganduglia Yamil

12 de septiembre de 2026

Notas del Autor

Mattia Federico, Ferrari Ulises, Del Pino Lucas y Orue Máximo, Departamento de Programación, Escuela de Educación Secundaria Técnica N.º 1 "Eduardo Ader".

La correspondencia relativa a este informe debe dirigirse a la dirección del grupo del proyecto *TeKnical May Cry* (TmC), E.E.S.T N.º 1, Vicente López, Buenos Aires.

Resumen

El presente informe técnico documenta el diseño, la arquitectura, el desarrollo y el control de calidad del proyecto educativo integrador titulado *TeKnical May Cry* (TmC), llevado a cabo durante el ciclo lectivo 2026. El objetivo principal consistió en unificar las competencias prácticas adquiridas en la especialidad de programación mediante la construcción de un videojuego en dos dimensiones (2D) implementado en el motor Unity utilizando C#, complementado por una plataforma web oficial maquetada en HTML5/CSS3/JavaScript y un módulo auxiliar escrito en C++. Se detalla el flujo de trabajo colaborativo basado en el sistema de control de versiones Git/GitHub, la organización de archivos e IDEs en sistemas de archivos Windows/Linux, y el cumplimiento estricto de las normativas institucionales de documentación técnica bajo norma APA v7. Los resultados acreditan un sistema funcional, optimizado y documentado.

Palabras clave: C#, Unity 2D, C++, HTML5, Git Bash, APA v7, Laboratorio de Programación.

Introducción

El desarrollo de software en las instituciones de educación técnica requiere la integración práctica de múltiples disciplinas computacionales. El proyecto *TeKnical May Cry* (TmC) surge como una respuesta académica orientada a consolidar las habilidades de

programación estructurada, programación orientada a objetos (POO), arquitectura de sistemas, diseño web responsivo y gestión de proyectos.

El trabajo se articula a partir de una doble vertiente técnica: por un lado, el desarrollo del motor interactivo y las mecánicas del juego; por el otro, la elaboración de la infraestructura web y la documentación técnica formal según el estándar APA v7 exigido para la entrega en la plataforma Moodle.

Objetivos del Proyecto

- **Objetivo General:** Desarrollar un entorno interactivo integrador compuesto por un videojuego 2D y un portal web técnico que apliquen conceptos de ingeniería de software y programación modular.
- **Objetivos Específicos:**
 1. Diseñar e implementar la lógica de interacción del personaje, sistemas de cámaras y minijuegos en lenguaje C# sobre el entorno Unity.
 2. Implementar subprogramas auxiliares y algoritmos de cómputo en C++ utilizando Visual Studio Code y el entorno MinGW.
 3. Establecer un flujo de trabajo continuo de control de versiones mediante Git y GitHub para la resolución de conflictos en equipo.
 4. Redactar el informe técnico y el diario de campo institucional respetando estrictamente los lineamientos de citación, jerarquía y formato de la norma APA (7.^a edición).

Marco Teórico y Estado del Arte

Entornos de Desarrollo e Infraestructura de Compiladores

El proceso de desarrollo de software moderno requiere distinguir entre Editores de Código Fuente e Entornos de Desarrollo Integrados (IDE). Durante el ciclo del proyecto, se utilizó Visual Studio Code como editor liviano extensible y Visual Studio Community para la compilación profunda en C#.

Configuración del Compilador MinGW y Extensión C/C++

Para el desarrollo del módulo de lógica **CALCULOAREA** escrito en C++, se configuró la suite de compilación MinGW (GCC/G++) asociada a Visual Studio Code. La automatización del proceso de compilación y depuración (*debugging*) se estableció mediante el archivo de configuración **tasks.json**.

Sistemas de Archivos y Almacenamiento

La gestión de almacenamiento en el sistema operativo depende de la arquitectura del sistema de archivos subyacente. En Windows, se operó primariamente sobre NTFS (*New Technology File System*), el cual administra estructuras mediante la Tabla Maestra de

Archivos (MFT, *Master File Table*), otorgando journaling, permisos NTFS y asignación por clústeres optimizados. Se evaluaron sistemas comparativos como FAT32, exFAT y ext4 para garantizar la portabilidad de los archivos ejecutables compilados de 64 bits.

Metodología y Desarrollo del Proyecto

Inspiración y Diseño Conceptual

El videojuego *TeKnical May Cry* combina mecánicas de dos títulos fundamentales de la industria:

1. **Diseño de Mapas y Navegación 2D:** Basado en la estructura top-down y mapas en cuadrícula de *Pokémon Rojo Fuego* (Nintendo, 2004). Se utilizaron componentes [Tilemap](#) y [Tilemap Collider 2D](#) para definir mallas de colisión exactas en salas y pasillos.
2. **Mecánicas de Minijuegos y Tensión:** Inspiradas en el loop de interacción y control de cámaras de *Five Nights at Freddy's* (Cawthon, 2014), gestionados mediante clases independientes en C# ([FNaFManager.cs](#) y [CameraSystem.cs](#)).

Desarrollo Backend en C# y C++

La arquitectura lógica se dividió en módulos independientes para prevenir el acoplamiento de código.

Implementación C# en Unity

La lógica del juego administra corrutinas para el manejo asíncrono de eventos en pantalla sin bloquear el hilo principal (*main thread*).

```
C#
// Ejemplo de estructura de control del minijuego en C#
public class FNaFManager : MonoBehaviour
{
    public float energyLevel = 100f;

    void Update()
    {
        if (energyLevel > 0)
        {
            energyLevel -= Time.deltaTime * 0.5f;
        }
    }
}
```

Figura 3

Ejecución de Terminal de Comandos durante la Compilación del Proyecto

Desarrollo Frontend Web

El frontend del proyecto web oficial fue estructurado en HTML5 semántico, estilizado con hojas de estilo en cascada (CSS3) utilizando flexbox y grillas adaptativas (*media queries*), e impulsado por scripts livianos en JavaScript para la interactividad de la galería y visores embebidos de documentos PDF.

Gestión de Versiones con Git y GitHub

Se utilizó Git Bash como consola principal para el control de versiones. Se aplicaron flujos de trabajo basados en ramas (*feature branching*), resolviendo conflictos de fusión (*merge conflicts*) en archivos de metadatos `.meta` generados por Unity. Las identidades del equipo se configuraron con credenciales fijas y tokens de acceso personal (PAT).

Resultados y Discusión

El desarrollo del proyecto *TeKnical May Cry* cumplió con el 100% de las especificaciones funcionales programadas en el cronograma. Los resultados principales incluyen:

1. **Rendimiento:** Estabilización de la tasa de cuadros por segundo (FPS) en el prototipo Unity mediante el reemplazo de llamadas redundantes en el método `Update()` por eventos C#.
2. **Portabilidad:** Verificación exitosa de compilación del subprograma `CALCULOAREA` en C++ tanto en arquitecturas x86 como x64.
3. **Documentación:** Publicación del informe técnico final y la carpeta de campo en la plataforma Moodle respetando los estándares de formato (tipografía Arial 11 / Times New Roman 12, interlineado doble, márgenes de 2.54 cm).

Conclusión

El proyecto *TeKnical May Cry* representó una experiencia integradora y enriquecedora dentro del plan de estudios del ciclo superior de la educación técnica. La combinación de desarrollo de videojuegos en C#, maquetación web y programación estructurada en C++ permitió afianzar conceptos teóricos en entornos de producción reales.

La superación de obstáculos técnicos —como la resolución de errores de referencia nula en scripts, la configuración de agentes SSH en Git Bash y la maquetación responsiva— fortaleció la autonomía, la perseverancia y la capacidad de resolución de problemas en equipo. La integración práctica entre teoría y ejecución reafirma la importancia del modelo de aprendizaje basado en proyectos.

Referencias

- Cawthon, S. (2014). *Five Nights at Freddy's* [Videojuego]. Scott Cawthon.

- Microsoft. (2023). *C# documentation: Basics of C# for beginners*. Microsoft Learn. <https://learn.microsoft.com/dotnet/csharp/>
- Nintendo. (2004). *Pokémon Rojo Fuego* [Videojuego]. Nintendo.
- Unity Technologies. (2025). *Unity User Manual 2022.3 LTS (Tilemaps and Physics 2D)*. Unity Documentation. <https://docs.unity3d.com/Manual/>
- Wikipedia contributors. (s. f.). *Pokémon rojo fuego y Pokémon verde hoja*. Wikipedia. https://es.wikipedia.org/wiki/Pok%C3%A9mon_rojo_fuego_y_Pok%C3%A9mon_verde_hoja
- Wikipedia contributors. (s. f.). *Scott Cawthon*. Wikipedia. https://es.wikipedia.org/wiki/Scott_Cawthon